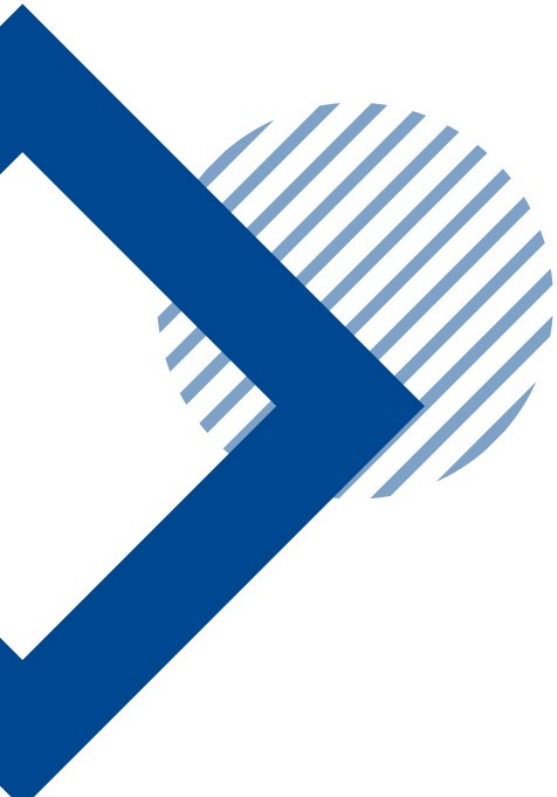
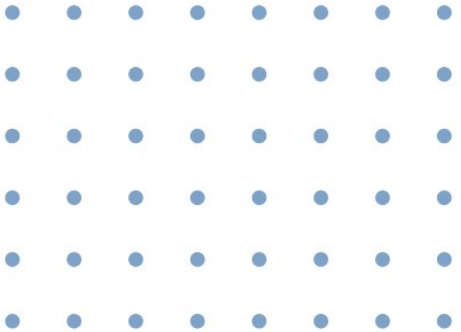
React: состояния и хуки

Интернет-программирование

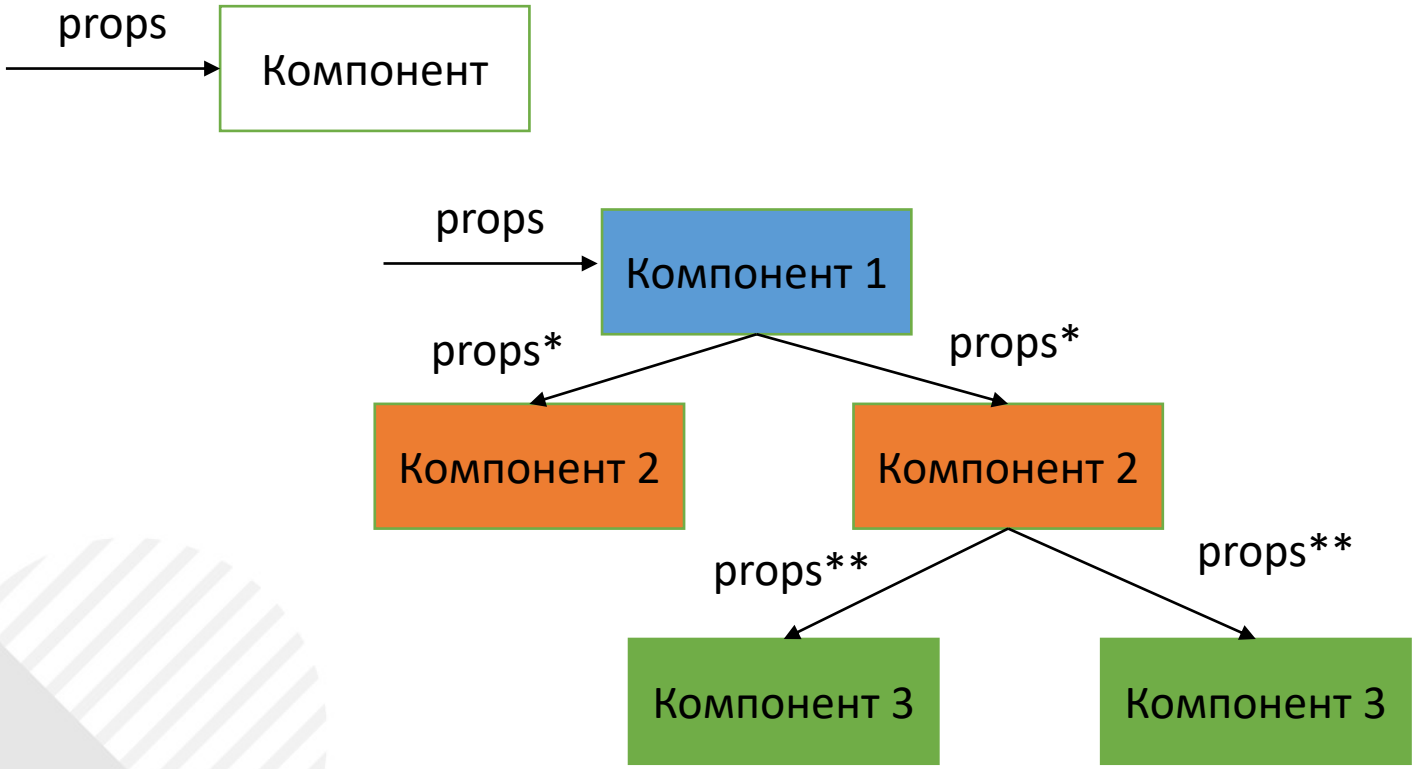






2022

Компонент React

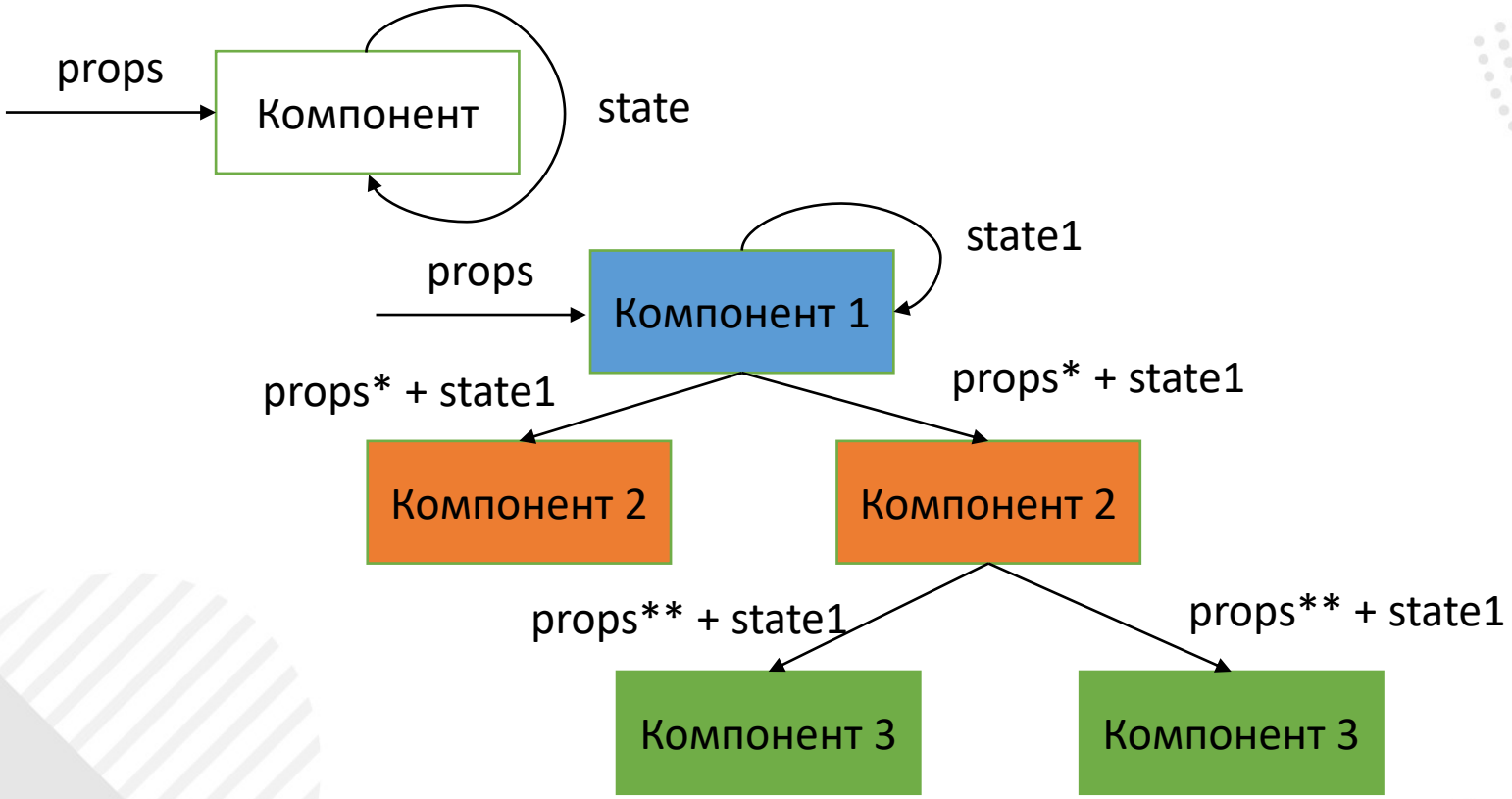


Компонент React

```
const Component1 = ({className, name}) => {
    return (
        <div className={className}>
            <Component2 title={name} />
        </div>
    );
};

const Component2 = ({title}) => {
    if (!title) {
        return;
    }
    return (
        <div>
            {title}
        </div>
    );
};
```

Состояние компонента React



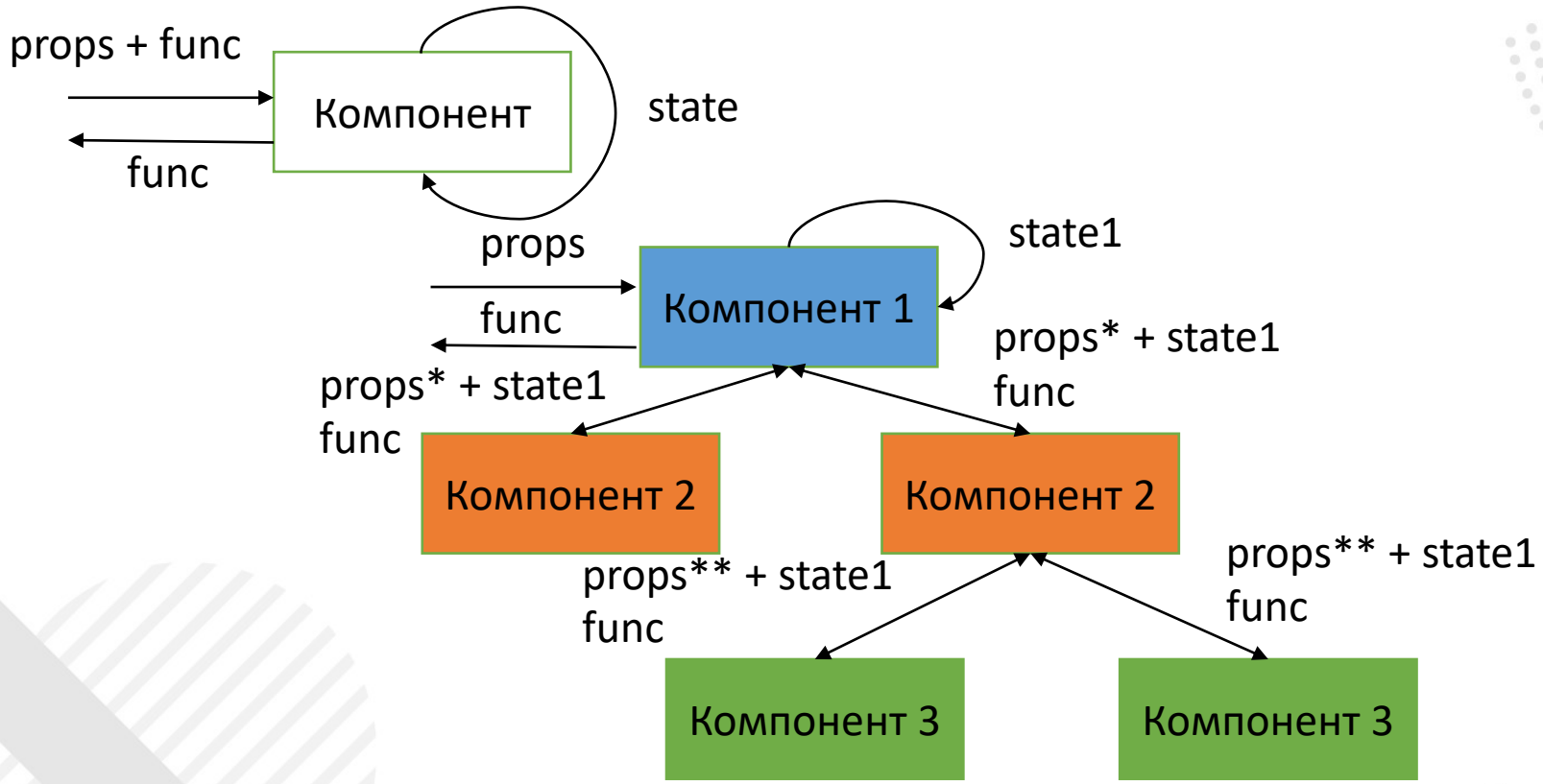
Состояние компонента React

```
const Component1 = ({className, items}) => {
    const [currentItem, setCurrentItem] = useState(0);
    return (
        <div className={className}>
            <Component2 items={items} current={currentItem} />
        </div>
    );
};

const Component2 = ({items, current}) => {
    const [visible, setVisible] = useState(true);
    return (
        <div>Current item ID is {current}</div>
       <button onClick={() => setVisible(!visible)}>{visible ? 'Показать' : 'Скрыть'}</button>
        <div visible={visible}>
        {
            items.map((item) =>
                <Component3 key={item.id} item={item} />
            )
        }
        </div>
    );
};
```



Функции как свойства компонента React

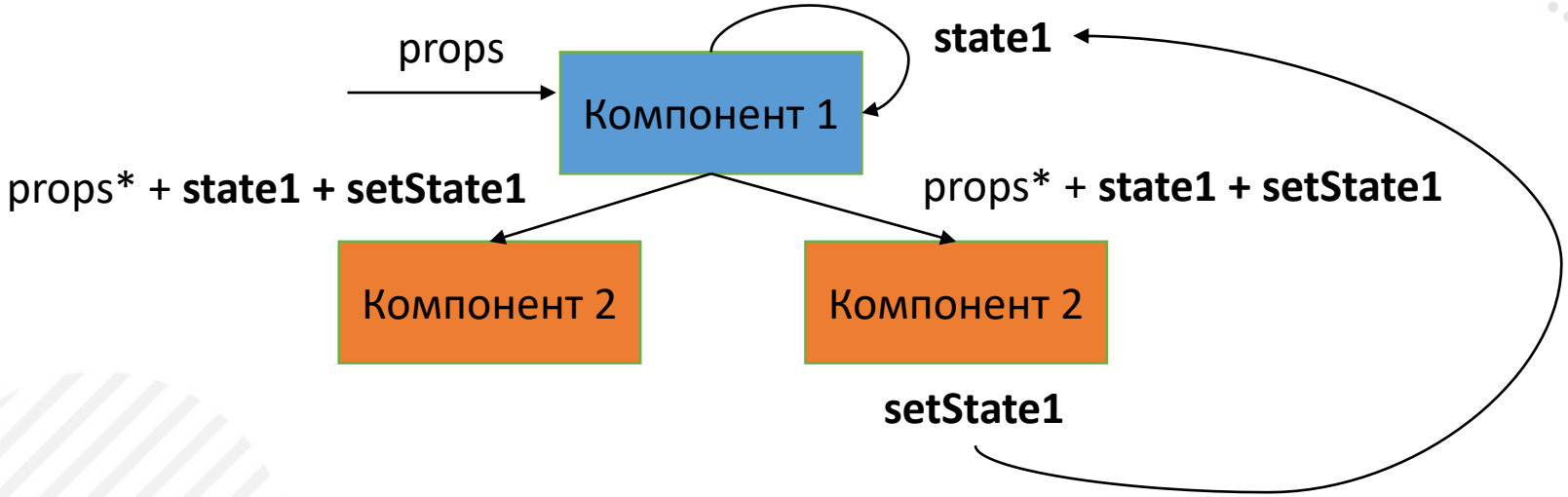


Функции как свойства компонента React

```
const Component1 = ({className, items}) => {
    const [currentItem, setCurrentItem] = useState(0);
    return (
        <div className={className}>
            <Component2 items={items} current={currentItem} setCurrent={setCurrentItem} />
            <Component2 items={items} current={currentItem} setCurrent={setCurrentItem} />
        </div>
    );
};

const Component2 = ({items, current, setCurrent}) => {
    const [visible, setVisible] = useState(true);
    return (
        <div>Current item ID is {current}</div>
       <button onClick={() => setVisible(!visible)}>{visible ? 'Показать' : 'Скрыть'}</button>
        <div visible={visible}>
        {
            items.map((item) =>
                <Component3 key={item.id} item={item} onClick={() => setCurrent(item.id)} />
            )
        }
         </div>
    );
};
```

Разделение состояния между компонентами



Пример приложения 1

Пример SPA приложения реализует следующие функции:

- Ввод и сохранение данных о товарах (тип, цена, количество, фото);
- Редактирование данных о товарах;
- Удаление данных о товарах;
- Фильтрация товаров по типу.

Для эмуляции бэкенда в стиле REST используется JSON Server.

Сущности:
- Товары (lines).
- Типы товаров (types).

Связи:
- Товар - многие к одному -> Тип товара.

Пример приложения 2

Товары:
- id – первичный ключ (integer);
- typeId – внешний ключ (integer);
- price – цена товара (integer);
- count – количество товара (integer);
- sum – общая стоимость всех товаров (integer);
- Image – изображение товара в base64 (string).

Типы товара:
- id – первичный ключ (integer);
- name – название типа (string).

Для организации связи используется столбец связывания typeId сущности Товары.

Пример приложения 3

REST API

Товары:
Create – POST: http://localhost:8081/lines + **Request Body JSON**
Read All – GET: http://localhost:8081/lines?**expand**=type&**typeId**={type}
Read – GET: http://localhost:8081/lines/{id}?**expand**=type
Update – PUT: http://localhost:8081/lines/{id} + **Request Body JSON**
Delete – DELETE: http://localhost:8081/lines/{id}

Типы товара:
Read All – GET: http://localhost:8081/types

Для работы с REST API используется библиотека **Axios.**

Axios работает аналогично fetch, но позволяет использовать перехватчики (interceptors).

Пример приложения 4

```
export const ApiClient = axios.create({
    baseURL: 'http://localhost:8081/',
    timeout: '3000',
    headers: {
        Accept: 'application/json',
    },
});

ApiClient.interceptors.response.use(responseHandler, responseErrorHandler);
```

Пример приложения 4

```
function responseHandler(response) {
    if (response.status === 200 || response.status === 201) {
        const data = response?.data;
        if (!data) {
            throw new HttpError('API Error. No data!');
        }
        return data;
    }
    throw new HttpError(`API Error! Invalid status code ${response.status}!`);
}

function responseErrorHandler(error) {
    if (error === null) {
        throw new Error('Unrecoverable error!! Error is null!');
    }
    toast.error(error.message, { id: 'AxiosError' });
    return Promise.reject(error.message);
}
```

Пример приложения 5

```
export class HttpError extends Error {
    constructor(message = '') {
        super(message);
        this.name = 'HttpError';
        Object.setPrototypeOf(this, new.target.prototype);
        toast.error(message, { id: 'HttpError' });
    }
}
```

Для вывода всплывающих сообщений используется библиотека React Hot Toast.

```
const App = ({ routes }) => {
  return (
    <>
      <Navigation routes={routes}></Navigation>
      <Container className='p-2' as='main' fluid>
        <Outlet />
      </Container>
      <Footer />
      <Toaster position='top-center' reverseOrder={true} />
    </>
  );
};
```

Пример приложения 6

```
class ApiService {
    constructor(url) {
        this.url = url;
    }

    async getAll(expand) {
        return ApiClient.get(`${this.url}${expand || ''}`);
    }

    async get(id, expand) {
        return ApiClient.get(`${this.url}/${id}${expand || ''}`);
    }

    async create(body) {
        return ApiClient.post(this.url, body);
    }

    async update(id, body) {
        return ApiClient.put(`${this.url}/${id}`, body);
    }

    async delete(id) {
        return ApiClient.delete(`${this.url}/${id}`);
    }
}
```

Пример приложения 7

Сервис для работы с REST API для сущности Товары
const LinesApiService = new ApiService('lines’);

Сервис для работы с REST API для сущности Типы товара
const TypesApiService = new ApiService('types’);

Сервисы нужны для сокрытия деталей реализации хранилища данных.

Можно заменить REST API на Local Storage и изменить код только в классе ApiService, остальной код изменен не будет.

Хуки в React 1

Хуки используются в React для выноса бизнес-логики из компонента пользовательского интерфейса.

Таким образом, появляется возможность использования архитектурного подхода разделения интересов (Separation of Concerns).

Бизнес-логика содержится в отдельном хуке и взаимодействует с компонентом пользовательского интерфейса только через параметры хука.

Параметры хука определяют интерфейс взаимодействия между компонентом пользовательского интерфейса и бизнес-логикой.

При использовании разделения интересов появляется возможность изменять содержимое хука без необходимости изменения компонента пользовательского интерфейса, но только при условии сохранения интерфейса (параметров хука).

Хуки в React 2

```
const Banner = () => {
    const [currentBanner, setCurrentBanner] = useState(0);
    const banners = [banner1, banner2, banner3];

    const getBannerClass = (index) => {
        return currentBanner === index ? 'banner-show' : 'banner-hide';
    };

    useEffect(() => {
        const bannerInterval = setInterval(
            () => {
                console.info('Banner changed');
                let current = currentBanner + 1;
                if (current === banners.length) {
                    current = 0;
                }
                setCurrentBanner(current);
            },
            5000,
        );
        return () => clearInterval(bannerInterval);
    });

    return (
        <div id="banner">
            {
                banners.map((banner, index) =>
                    <img key={index} className={getBannerClass(index)} src={banner} alt={`banner${index}`} />)
            }
        </div >
    );
};
```

Хуки в React 3

```
const Banner = () => {
    const { banners, getBannerClass } = useBannerHook();
    return (
        <div id='banner'>
            {
                banners.map((banner, index) =>
                    <img key={index} className={getBannerClass(index)} src={banner}
                        alt={`banner${index}`} />)
            }
        </div >
    );
};
```

Хуки в React 4

```
const useBannerHook = () => {
    const [currentBanner, setCurrentBanner] = useState(0);
    const banners = [banner1, banner2, banner3];

    const getBannerClass = (index) => {
        return currentBanner === index ? 'banner-show' : 'banner-hide';
    };

    useEffect(() => {
        const bannerInterval = setInterval(
            () => {
                console.info('Banner changed');
                let current = currentBanner + 1;
                if (current === banners.length) {
                    current = 0;
                }
                setCurrentBanner(current);
            },
            5000,
        );
        return () => clearInterval(bannerInterval);
    }, []);

    return {
        banners,
        getBannerClass,
    };
};
```

Хуки в React 5

```
const useLines = (typeFilter) => {
    const [linesRefresh, setLinesRefresh] = useState(false);
    const [lines, setLines] = useState([]);
    const handleLinesChange = () => setLinesRefresh(!linesRefresh);

    const getLines = async () => {
        let expand = '?_expand=type';
        if (typeFilter) {
            expand = `${expand}&typeId=${typeFilter}`;
        }
        const data = await LinesApiService.getAll(expand);
        setLines(data ?? []);
    };

    useEffect(() => {
        getLines();
    }, [linesRefresh, typeFilter]);

    return {
        lines,
        handleLinesChange,
    };
};
```

Хуки в React 5

```
const Lines = () => {
    const { types, currentFilter, handleFilterChange } = useTypeFilter();
    const { lines, handleLinesChange } = useLines(currentFilter);
    return (
        <>
            <Select className='mt-2' values={types} label='Фильтр по товарам'
                value={currentFilter} onChange={handleFilterChange} />
            <LinesTable>
                {
                    lines.map((line, index) =>
                        <LinesTableRow key={line.id}
                            index={index} line={line}
                            onDelete={() => showDeleteModal(line.id)}
                            onEdit={() => showFormModal(line.id)}
                            onEditInPage={() => showEditPage(line.id)}
                        />)
                }
            </LinesTable>
        </>
    );
};
```

Хуки в React 6

```
const useTypeFilter = () => {
    const filterName = 'type';

    const [searchParams, setSearchParams] = useSearchParams();

    const { types } = useTypes();

    const handleFilterChange = (event) => {
        const type = event.target.value;
        if (type) {
            searchParams.set(filterName, event.target.value);
        } else {
            searchParams.delete(filterName);
        }
        setSearchParams(searchParams);
    };

    return {
        types,
        currentFilter: searchParams.get(filterName) || '',
        handleFilterChange,
    };
};
```

Хуки в React 7

```
const useTypes = () => {
    const [types, setTypes] = useState([]);

    const getTypes = async () => {
        const data = await TypesApiService.getAll();
        setTypes(data ?? []);
    };

    useEffect(() => {
        getTypes();
    }, []);

    return {
        types,
    };
};
```

Модальные окна 1

```
const ModalForm = ({
   show, title, validated, onSubmit, onClose, children,
}) => {
   return createPortal(
      <Modal show={show} backdrop='static' onHide={() => onClose()}>
         <Modal.Header className='pt-2 pb-2 ps-3 pe-3' closeButton>
            <Modal.Title>{title}</Modal.Title>
         </Modal.Header>
         <Form className='m-0' noValidate validated={validated} onSubmit={onSubmit}>
            <Modal.Body>
               {children}
            </Modal.Body>

            <Modal.Footer className='m-0 pt-2 pb-2 ps-3 pe-3 row justify-content-center'>
               <Button variant='secondary' className='col-5 m-0 me-2'
                  onClick={() => onClose()}>
                  Отмена
               </Button>
               <Button variant='primary' className='col-5 m-0 ms-2' type='submit'>
                  Сохранить
               </Button>
            </Modal.Footer>
         </Form>
      </Modal>,
      document.body,
   );
};
```

Модальные окна 2

```
const ModalConfirm = ({
   show, title, message, onConfirm, onClose,
}) => {
   return createPortal(
      <Modal show={show} backdrop='static' onHide={() => onClose()}>
         <Modal.Header className='pt-2 pb-2 ps-3 pe-3' closeButton>
            <Modal.Title>{title}</Modal.Title>
         </Modal.Header>

         <Modal.Body>
            {message}
         </Modal.Body>

         <Modal.Footer className='m-0 pt-2 pb-2 ps-3 pe-3 row justify-content-center'>
            <Button variant='secondary' className='col-5 m-0 me-2'
               onClick={() => onClose()}>
               Нет
            </Button>
            <Button variant='primary' className='col-5 m-0 ms-2'
               onClick={() => onConfirm()}>
               Да
            </Button>
         </Modal.Footer>
      </Modal>,
      document.body,
   );
};
```

Модальные окна 3

```
const useModal = () => {
    const [showModal, setShowModal] = useState(false);

    const showModalDialog = () => {
        setShowModal(true);
    };

    const hideModalDialog = () => {
        setShowModal(false);
    };

    return {
        isModalShow: showModal,
        showModal: showModalDialog,
        hideModal: hideModalDialog,
    };
};
```

Модальные окна 4

```
const {
        isDeleteModalShow,
        showDeleteModal,
        handleDeleteConfirm,
        handleDeleteCancel,
 } = useLinesDeleteModal(handleLinesChange);

<ModalConfirm show={isDeleteModalShow}
              onConfirm={handleDeleteConfirm} onClose={handleDeleteCancel}
              title='Удаление' message='Удалить элемент?' />


<LinesTableRow key={line.id}
                       index={index} line={line}
                       onDelete={() => showDeleteModal(line.id)}
                       onEdit={() => showFormModal(line.id)}
                       onEditInPage={() => showEditPage(line.id)}
                    />)
```

Модальные окна 4

```
const useLinesDeleteModal = (linesChangeHandle) => {
    const { isModalShow, showModal, hideModal } = useModal();
    const [currentId, setCurrentId] = useState(0);

    const showModalDialog = (id) => {
        showModal();
        setCurrentId(id);
    };

    const onClose = () => {
        hideModal();
    };

    const onDelete = async () => {
        await LinesApiService.delete(currentId);
        linesChangeHandle();
        toast.success('Элемент успешно удален', { id: 'LinesTable' });
        onClose();
    };

    return {
        isDeleteModalShow: isModalShow,
        showDeleteModal: showModalDialog,
        handleDeleteConfirm: onDelete,
        handleDeleteCancel: onClose,
    };
};
```

Формы и управляемые компоненты 1

```
const [validated, setValidated] = useState(false);

const [formData, setFormData] = useState({
        lastname: '',
        firstname: '',
});

const handleSubmit = (event) => {
        const form = event.currentTarget;
        event.preventDefault();
        event.stopPropagation();
        if (form.checkValidity() !== false) {
            console.log(formData);
        }
        setValidated(true);
};

const handleChange = (event) => {
        const inputName = event.target.name;
        const inputValue = event.target.type === 'checkbox' ? event.target.checked : event.target.value;
        setFormData({
            ...formData,
            [inputName]: inputValue,
        });
};
```

Формы и управляемые компоненты 2

```
return (
      <div className='row justify-content-center'>
        <Form className='col-md-6 m-0' noValidate validated={validated}
          onSubmit={handleSubmit}>
          <Form.Group className='mb-2' controlId='lastname'>
            <Form.Label>Фамилия</Form.Label>
            <Form.Control type='text' name='lastname' required
              value={formData.lastname} onChange={handleChange} />
          </Form.Group>
          <Form.Group className='mb-2' controlId='firstname'>
            <Form.Label>Имя</Form.Label>
            <Form.Control type='text' name='firstname' required
              value={formData.firstname} onChange={handleChange} />
          </Form.Group>
          <div className='text-center'>
            <Button className='w-50' variant='primary' type='submit'>Отправить</Button>
          </div>
        </Form>
      </div>
);
```

Спасибо за внимание!

Литература:

1. Updating Objects in State - https://react.dev/learn/updating-objects-in-state
2. Sharing State Between Components - https://react.dev/learn/sharing-state-between-components
3. React Hot Toast - https://react-hot-toast.com
4. Axios - https://axios-http.com
5. Axios & Error handling like a boss - https://dev.to/mperon/axios-error-handling-like-a-boss-333d
6. Separation of Concerns in React –How to Use Container and Presentational Components - https://www.freecodecamp.org/news/separation-of-concerns-react-container-and-presentational-components/
7. Separation of concerns in React and React Native - https://dev.to/sathishskdev/separation-of-concerns-in-react-and-react-native-45b7
8. React Bootstrap - https://react-bootstrap.netlify.app
9. React Bootstrap Icons - https://github.com/ismamz/react-bootstrap-icons
10. JSON Server - https://www.npmjs.com/package/json-server



2022